



Name: Shayan Umar

Designation: AI intern

Department: Artificial Intelligence

Date: 29-8-2025

Table of Contents

1. Theoretical Understanding:	3
1.1 Image Classification:	3
1.2 Rules of Machine Learning:	6
1.3 People + AI GuideBook:	10
1.4 Text Classification:	17
1.5 Good Data Analysis:	22
1.6 Deep Learning Tuning Play-book:	24
1.7 Introduction to Responsible AI:	34
1.8 Adversarial Testing for Generative AI:	36
2. Self-Directed Exploration:	38
2.1 Unsloth for Finetuning Large Language Models:	38
3. Challenges and Approaches:	42
4. Result and Analysis:	43
5. Conclusion:	46
6. References:	47

1. Theoretical Understanding:

1.1 Image Classification:

A **Convolutional Neural Network (CNN)** is a type of neural network that's especially good at working with **images**.

Instead of humans manually telling it what features (edges, shapes, textures) to look for, the CNN learns those features by itself from raw pixel data.

Think of it as a **feature extractor + classifier**:

1. **Feature extractor**: Finds useful details in an image (like edges, corners, shapes).
 2. **Classifier**: Decides what the image represents (cat, dog, car, etc.).
-

Main Components of a CNN

1. Convolution Layer

- Purpose: **Detect features** (edges, corners, colors, textures).
- How it works: Small filters (tiny grids like 3×3 or 5×5) slide over the image.
- Each filter focuses on a certain type of feature.
- Output = a **feature map** (new image-like grid showing where that feature appears).
- During training, the CNN **learns the best filter values** automatically.

Example: One filter might detect horizontal edges, another detects circles, another detects color contrasts.

2. Activation Function (ReLU)

- Purpose: **Introduce non-linearity** (real-world data is not just straight lines).
 - ReLU (Rectified Linear Unit) replaces negative values with 0, keeps positives the same.
 - Without this step, the network would just be doing simple linear math and couldn't capture complex patterns.
-

3. Pooling Layer

- Purpose: **Reduce the size of the data** but keep the important info.
 - Most common = **Max Pooling**: Take the maximum value from each small region (like picking the strongest signal).
 - Benefits:
 - Makes computations faster.
 - Helps the model focus on the most important features.
 - Reduces overfitting (less chance the model memorizes exact positions of features).
-

4. Fully Connected Layers (Dense Layers)

- After convolution + pooling, we flatten everything into a **long vector of numbers**.
- Fully connected layers work just like in regular neural networks.
- Their job = **combine all the features** and decide the final class (cat, dog, etc.).
- The last layer often uses **Softmax**, which outputs probabilities for each class.

Example: [Cat: 0.9, Dog: 0.05, Car: 0.05].

Preventing Overfitting

As with any machine learning model, a key concern when training a convolutional neural network is *overfitting*: a model so tuned to the specifics of the training data that it is unable to generalize to new examples. Two techniques to prevent overfitting when building a CNN are:

- **Data augmentation**: artificially boosting the diversity and number of training examples by performing random transformations to existing images to create a set of new variants (see Figure 7). Data augmentation is especially useful when the original training data set is relatively small.
- **Dropout regularization**: Randomly removing units from the neural network during a training gradient step.

Leveraging Pre-Trained Models

Training CNNs from scratch needs massive data and time. Instead, you can use **pretrained models** (trained on huge datasets) and apply **feature extraction**: take the high-level features (like shape, color, texture) from a pretrained model's convolution layers, and feed them into your own classifier for your specific task. To boost performance, you can also **fine-tune** the pretrained weights slightly so they adapt better to your dataset.

1.2 Rules of Machine Learning:

Before Using Machine Learning

- **Rule #1:** Start simple. Don't launch with ML unless you truly need it. Heuristics (rules of thumb) and even human editing can go a long way.
 - **Rule #2:** Set up **metrics early**. Measure everything you can — it helps you understand your system and gather historical data.
 - **Rule #3:** Prefer ML over **complex heuristics**, since heuristics become hard to maintain, while ML models are easier to update.
-

Building Your First ML Pipeline

- **Rule #4:** Keep the first model simple; focus on **infrastructure reliability** (data flow, feature delivery, integration).
 - **Rule #5:** Test infrastructure **independently** of ML to ensure correctness in data and model usage.
 - **Rule #6:** Watch out for **dropped or missing data** when copying pipelines.
 - **Rule #7:** Convert existing heuristics into **features** rather than discarding them — they often encode valuable intuition.
-

Monitoring and Maintenance

- **Rule #8:** Know how **fresh** your model needs to be (daily, weekly, etc.), depending on how fast the data changes.
 - **Rule #9:** Do **sanity checks** before exporting models to production.
 - **Rule #10:** Watch for **silent failures** (e.g., stale tables, missing data) — they degrade performance gradually.
 - **Rule #11:** Assign **owners and documentation** to feature columns so knowledge isn't lost.
-

Defining Objectives

- **Rule #12:** Don't overcomplicate objectives at first — simple ones often improve all metrics.
 - **Rule #13:** Choose an **observable and attributable** metric (like clicks or downloads), not vague goals (like “user happiness”).
 - **Rule #14:** Start with **interpretable models** (e.g., logistic regression) for easier debugging.
 - **Rule #15:** Separate **spam filtering** from **quality ranking** — they behave differently and evolve at different speeds.
-

In essence:

Start with simple heuristics and strong metrics → build a robust pipeline with a simple model → carefully monitor data and freshness → define clear, measurable objectives → evolve toward ML, using heuristics as features and separating concerns like spam vs. quality.

ML Phase II: Feature Engineering

Once you have a working end-to-end ML system (Phase I), the focus shifts to **adding and refining features**. This is where most easy gains happen.

Key Rules:

- **Rule #16 – Launch and iterate:** Don't treat any model as final. Keep improving by adding new features, tweaking objectives, and refreshing pipelines.
- **Rule #17 – Prefer observed features:** Start with features directly measured (clicks, views, etc.), not “learned” features (like clusters or embeddings) — they're easier to debug and more stable.
- **Rule #18 – Use generalizable content features:** Features like co-views, ratings, or actions in other contexts help new/unseen content succeed.
- **Rule #19 – Don't fear specific features:** With enough data, even highly specific features (like a query–document ID pair) can be useful. Regularization will filter out weak ones.
- **Rule #20 – Create new features from old ones:**
 - *Discretization:* Turn continuous variables into buckets (e.g., age groups).
 - *Crosses:* Combine categorical features (e.g., {male, female} × {US, Canada}). Needs lots of data to avoid overfitting.
- **Rule #21 – Match feature complexity to data size:**

- Few examples → simple features (like TF-IDF).
 - Millions → intersected features with regularization.
 - Billions → complex crosses and embeddings.
 - **Rule #22 – Remove unused features:** Keep pipelines clean. Only keep impactful features.
-

Human Analysis

Not everything is automated — humans must inspect and guide the process.

- **Rule #23 – You aren't a typical user:** Engineers are biased. Use real users, crowdworkers, or A/B tests.
 - **Rule #24 – Measure differences between models:** Compare old vs. new outputs. If results barely differ, don't expect big improvements.
 - **Rule #25 – Utility > raw predictive power:** The usefulness of predictions (ranking quality, user satisfaction) matters more than abstract accuracy.
 - **Rule #26 – Learn from errors:** Look at false positives/negatives, add features to fix them.
 - **Rule #27 – Quantify frustrations:** Turn complaints ("too many gag apps") into measurable signals.
 - **Rule #28 – Short-term ≠ long-term:** A system that looks fine in testing may stagnate because it doesn't promote new content.
-

Training–Serving Skew

Models often perform differently in training vs. production. Causes: data drift, pipeline mismatches, feedback loops.

- **Rule #29 – Log serving-time features:** Use the exact same features for training as you do at serving.
- **Rule #30 – Use importance-weighting when sampling:** Don't just drop data randomly.

- **Rule #31 – Be cautious with joins:** Tables can change between training and serving. Snapshot or log instead.
 - **Rule #32 – Re-use code:** Sharing as much code as possible between training and serving avoids bugs.
 - **Rule #33 – Test on future data:** Always validate on “next day” or later data, not just training splits.
 - **Rule #34 – For filtering, favor clean data:** Even if you show a tiny random sample of “blocked” items, it gives unbiased training data.
 - **Rule #35 – Watch for skew in ranking problems:** Changing ranking changes the data you’ll get in the future — design features accordingly.
 - **Rule #36 – Be careful with positional bias:** Train with position features, but don’t use them directly at serving.
 - **Rule #37 – Measure training/serving skew explicitly.**
-

ML Phase III: Refinement & Complex Models

At some point, **easy improvements slow down**. Gains get smaller, trade-offs between metrics appear, and you need more sophistication.

Key Rules:

- **Rule #38 – Fix objectives, not just features:** If metrics plateau, the real issue may be that your objective no longer matches business goals.
- **Rule #39 – Launch decisions $\neq$ model accuracy:** Real-world success depends on broader metrics (DAU, revenue, long-term engagement), not just loss functions.
- **Rule #40 – Keep ensembles simple:** Base models + a simple combiner is fine. Avoid deep chains of stacked models.
- **Rule #41 – Look for new information sources:** When refinements stop helping, add qualitatively new features (user history, external knowledge, deep learning).

1.3 People + AI GuideBook:

Step 1: Start with People, Not Just Tech

- **People-first approach:** Identify *real user problems* → validate assumptions about *who* has the problem, *where*, and *why*.
 - **Tech-first approach:** Start from AI capabilities and explore what problems they could solve (e.g., adding a safety classifier to improve UX).
Both are valid, but balance is key. Always ground decisions in **user research and context**.
-

Step 2: Decide if AI Adds Unique Value

- Ask: Does AI actually improve the user experience?
 - Not all solutions benefit from AI; some may even get worse if over-automated.
 - Consider:
 - **Stakes** (Is accuracy crucial, or is creativity allowed?)
 - **User context** (How/when will people use this system?)
 - **Cost** (AI may add expense and complexity).
-

Step 3: Understand the Problem Deeply

- Clearly describe and validate the problem with diverse users.
 - Talk to “extreme users” (people with very different needs/contexts).
 - Look for **sub-problems** AI must solve (across age, location, literacy, disability, etc.).
 - Remember: Problems evolve, and people experience them differently.
-

Step 4: Specification Alignment (Matching AI Behavior to User Intent)

- **Primary Goal:** What the user really wants solved (e.g., “I want varied running trails so I stay motivated”).

- **Sub-goals:** Steps users must take before/while solving the big goal (gear, hydration, warmup, etc.).
- **Underspecification:** Users may not state everything (fitness level, safety concerns). AI must handle these gaps.
- **Optimization trade-offs:** Improving one aspect (fun) could harm another (safety).

Risk: If misaligned, AI may optimize the wrong thing (reward hacking).

Step 5: Design for Diversity & Fairness

- AI performance can vary across groups (age, gender, language, socioeconomic status, etc.).
 - Lack of diverse data = biased outputs → risks excluding or harming some users.
 - Mitigation:
 - Include diverse voices in design.
 - Use fairness benchmarks (like WinoBias).
 - Test outputs across different user groups.
-

Step 6: Consider AI Types

- **Traditional AI:** classification, regression, recommenders. Good for efficiency, prediction, detection (fraud, face recognition, etc.).
 - **Generative AI:** open-ended tasks (text, images, code). Requires more care because outputs are unpredictable and can reflect bias or inaccuracies.
-

Step 7: Study How People Currently Solve the Problem

- Map existing workflows.
- Identify which steps can be automated or enhanced with AI.
- Learn from existing AI tools (successes & shortcomings).
- Test ideas quickly with prototypes or “Wizard of Oz” tests (simulate AI manually before building).

Balance Automation and Augmentation

Automation

- **Definition:** The AI does a task with little to no human intervention.
- **Best for:** Repetitive, low-value, or time-consuming tasks where oversight isn't critical.
- **Examples:** Sorting photos into albums, filtering spam emails.
- **Benefits:**
 - Saves time and effort
 - Reduces cognitive load
 - Increases safety by removing risk for people
- **Key measure of success:** higher efficiency, performance, and satisfaction.

Augmentation

- **Definition:** AI works with people to **amplify their capabilities** rather than replacing them.
- **Best for:** Tasks where human judgment, creativity, or nuance is valuable.
- **Examples:**
 - A travel chatbot suggesting personalized trip options
 - AI assistants helping doctors spot anomalies in scans (but not making the final call).
- **Benefits:**
 - Gives people new insights
 - Helps them make better decisions
 - Improves efficiency without fully removing control
- **Key measure of success:** more productivity, creativity, and user empowerment.

Key Design Considerations

- **Balance:** Some tasks may need a mix of automation and augmentation.
- **User trust:** People may over-rely on automation (automation bias) or stop paying attention when automation is present (automation complacency). For high-stakes tasks, human oversight is essential.

- **User understanding:** People often don't fully know what AI can/can't do. Products should guide them, set expectations, and build trust.

Define Interaction Design Policies

What are Interaction Design Policies?

They're a set of **criteria and guardrails** that define:

- What AI in your product *should* do (acceptable actions).
- What AI *shouldn't* do (unacceptable actions).
- How much uncertainty is safe to show to users.
- What risks/vulnerabilities users might face.

They make sure AI responses are **safe, useful, and aligned with user needs**, especially in open-ended generative AI scenarios.

Anatomy of a Design Policy

1. **Critical Moments** → When users interact with AI in ways that shape their trust (e.g., chatbot answers, medical suggestions).
 2. **Acceptable Actions** → Tasks AI should do well and consistently.
 3. **Unacceptable Actions** → Tasks AI must not attempt, even if asked.
 4. **Thresholds of Uncertainty** → Minimum quality criteria before showing results.
 5. **Vulnerabilities** → Risks from AI errors and their consequences.
-

Why They Matter

- Users form trust at **critical moments**, so failures here are costly.
 - AI errors are inevitable → policies help balance **precision vs recall**:
 - **Precision-focused** → Fewer but more accurate results (safer, but limited).
 - **Recall-focused** → More results, including weaker ones (broader, but riskier).
 - Policies must balance **usefulness, safety, and inclusivity**.
-

Examples

- **Low-risk (Run app):**
 - Precision → Show only trails users are very likely to enjoy.
 - Recall → Suggest more trails, even if some don't fit preferences (good for discovery).
 - **High-risk (Cancer detection):**
 - Precision is critical → Better to miss some weak signals than risk false positives that could cause harm.
-

How to Craft Policies

1. **Identify critical moments** (via journey maps, user flows).
2. **Collaborate across teams** (UX, product, AI/engineering).
3. **Define success criteria** (both qualitative + quantitative).
4. **Prototype & test with users** before full model deployment.
5. **Iterate and evaluate regularly** → user needs, cultural context, and risks may change.

Prepare For Potential Pitfalls

Why second-order effects matter

- **First-order consequences** → Immediate outcomes of AI use (e.g., students use AI to draft essays).
 - **Second-order consequences** → Ripple effects of those outcomes (e.g., over-reliance on AI reduces critical thinking).
 - These are **hard to predict** but not impossible—you can spot signals of change by studying user behavior, social adoption, and cultural context.
-

How to plan for potential pitfalls

1. **Resilient product visions**
Define the long-term future you want to enable—and what you want to avoid.
 2. **Proactive mitigations**
Anticipate risks and design strategies to reduce harm before launch.
 3. **Signals of change**
Track shifts in user expectations, social behavior, or regulations that could redefine “acceptable” AI actions.
 4. **AI principles and values**
Clarify applications you *will* and *won't* pursue.
 5. **Red-teaming**
Actively test your AI for security, fairness, and bias risks.
-

Types of harms to watch for

- **Allocative harms** → Unequal access to resources or opportunities.
 - **Quality-of-service harms** → AI works better for some groups than others.
 - **Representational harms** → Reinforcing stereotypes or bias in generated outputs.
 - **Social/interpersonal harms** → Negative ripple effects across communities and relationships.
-

Practical strategies

- **Expand perspectives:** Bring in diverse voices to anticipate downstream harms.
 - **Plan for behavior change:** Users will adapt and find new uses, so policies must evolve.
 - **Monitor continuously:** Track product feedback and key metrics post-launch.
 - **Iterate design policies:** Regularly update them to reflect shifting user intent, cultural expectations, and regulation.
-

Example monitoring triggers

- If **20%+ of users reject AI recommendations**, recheck your ML model.
- If **60% download but never use** the app, revisit marketing & onboarding.

- If **engagement is high but task completion low**, talk to users and adjust notifications or UX.

1.4 Text Classification:

Introduction

Train a model to predict whether a review is **positive** or **negative**. The classic IMDB dataset has **50k labeled reviews** ($\approx 25k$ train / $25k$ test, class-balanced). We'll follow a standard ML lifecycle that keeps you honest about quality, reproducibility, and deployment.

Step 1: Gather Data

What to do

- Use a vetted dataset (e.g., IMDB) or collect your own reviews with labels.
- Ensure **labels** are reliable (human-rated or clear heuristics).
- Record **metadata**: review ID, source, date, language, rating (but don't leak it).
- Split early into **train/validation/test** (stratified), e.g., 80/10/10 of the training portion.

Why it's necessary

- Good data > fancy models. If labels are noisy or splits leak information, everything downstream is misleading.
 - Early, clean splits prevent accidental **data leakage** and give you a trustworthy yardstick.
-

Step 2: Explore Your Data

What to do

- Examine **class balance**, **review length** distribution, and top **tokens/bi-grams**.
- Look for **duplicates**, boilerplate (e.g., "This review may contain spoilers"), or HTML artifacts.
- Spot **label noise** and edge cases (sarcasm, mixed sentiment, very short reviews).
- Build **quick baselines** (e.g., majority class, lexicon score) to set a floor.

Why it's necessary

- EDA reveals pitfalls (leakage, imbalance, junk text) and informs preprocessing choices (max sequence length, cleaning rules).

- Baselines ensure your model beats “naïve” strategies.
-

Step 2.5: Choose a Model

Step 1: Why this matters

When we train a machine learning model for text classification (like sentiment analysis or topic classification), we need to decide:

1. **How to turn text into numbers** (so the computer can understand it).
2. **Which type of model to use** (simple or advanced).

The choice depends on the size of the dataset and the average length of the text samples.

Step 2: The key idea

We look at the **ratio**:

Ratio = $\frac{\text{Number of samples}}{\text{Average number of words per sample}}$

- If the **ratio is small (<1500)** → the text samples are *longer compared to how many examples we have*.
Use **n-grams + MLP** (a simple neural network).
Reason: Simple models work just as well here, and they’re faster to train.
 - If the **ratio is large (>=1500)** → we have many samples, but each is fairly short.
Use **sequence models** (like CNNs or RNNs) that can capture word order.
Reason: These models benefit from lots of examples and can understand sequences better.
-

Step 3: How the preprocessing changes

- **For small ratio (<1500) → MLP (Option A):**
 - Break text into **n-grams** (like "very good", "not bad").
 - Convert them into numeric vectors.
 - Pick the **20K most important features**.
 - Train a **simple multi-layer perceptron (MLP)** model.
- **For large ratio (>=1500) → CNN (Option B):**

- Split text into **words**.
 - Select the **20K most frequent words**.
 - Turn each text into a **sequence of word IDs**.
 - Train a **CNN (sepCNN)** model.
 - If ratio is **<15K**, you can improve results by using **pretrained embeddings** (like GloVe or Word2Vec).
-

Step 4: Example (IMDb Reviews dataset)

- IMDb dataset: ratio $\approx$ **144**.
 - Since **144 < 1500**, we fall into the **small ratio** case.
Best choice: **n-grams + simple MLP model**.
-

In short:

- If dataset is small/medium or text samples are long $\rightarrow$ use **n-grams + simple MLP**.
 - If dataset is large and text samples are short $\rightarrow$ use **sequence models (CNN/RNN)**.
-

Step 3: Prepare Your Data

What to do

- **Cleaning** (light-touch): strip HTML, normalize whitespace; avoid over-cleaning (don't remove all punctuation—transformers learn from it).
- **Tokenization**:
 - Classic: word/character n-grams + TF-IDF.
 - Transformers: use the **pretrained tokenizer** (WordPiece/BPE); keep case if the model is cased.
- **Sequence handling**: set **max_length** (e.g., 256–512); **truncate/pad** consistently.
- **Vocabulary & OOV**: handled by TF-IDF or the pretrained tokenizer automatically.
- **Shuffling & batching**: shuffle train; keep val/test fixed.

- Persist the exact **preprocessing pipeline** (versions, seeds).

Why it's necessary

- Consistent preprocessing prevents **training/serving skew**.
 - Length choices affect memory/latency; tokenizer choice must match the pretrained model.
-

Step 4: Build, Train, and Evaluate Your Model

What to do

- **Loss/metrics**: Binary cross-entropy (or softmax CE) with **Accuracy, Precision/Recall/F1, ROC-AUC**; inspect **confusion matrix**.
- **Regularization**: dropout/weight decay; early stopping on validation loss/F1.
- **Optimization**: Adam/AdamW; learning-rate scheduling (warmup + decay for transformers).
- **Evaluation protocol**: keep a **held-out test** untouched until the end.
- **Error analysis**: read misclassified reviews; categorize by length, sarcasm, domain shift.

Why it's necessary

- Proper metrics catch different failure modes (e.g., false positives vs. false negatives).
 - Error analysis creates **feature/model ideas** far faster than blind tweaking.
-

Step 5: Tune Hyperparameters

What to do

- **Classical models**: C (regularization), n-gram range, min_df, max_features.
- **Neural/transformers**: learning rate, batch size, epochs, max_length, dropout, weight decay, warmup steps.
- Use **random search** or **Bayesian optimization** over sensible ranges; stop based on **validation**.
- Track experiments (config, seed, metrics) for reproducibility.

Why it's necessary

- The same architecture can vary widely by hyperparams; structured tuning yields predictable gains without overfitting.
-

Step 6: Deploy Your Model

What to do

- Package **preprocessing + model** together (e.g., a single pipeline in a Docker image).
- Expose via **API** (REST/gRPC) or batch job; add **request timeouts** and **rate limits**.
- Add **observability**: latency, throughput, error rates; optional confidence scores.
- Set up **monitoring** for **data drift** (token distributions, review lengths), **performance drift** (drop in F1), and **feedback loops**.
- Document model card: data sources, limitations (sarcasm, new slang), known biases.

Why it's necessary

- Consistent inference + monitoring keeps quality from quietly degrading, and makes rollback & iteration safe.
-

Conclusion

- Start simple, verify the pipeline, then scale model complexity.
- Let **error analysis and metrics** guide improvements.
- Treat deployment and monitoring as first-class: sentiment and language **drift over time**.

1.5 Good Data Analysis:

Overview

- Data analysis is powerful but error-prone, especially with large, high-dimensional datasets like those at Google.
 - Good analysts build credibility by being careful, methodical, and skeptical.
 - Document covers three aspects of good analysis: **Technical, Process, and Mindset**.
-

Technical: How to Examine Data

- **Look at distributions:** Go beyond mean/median—use histograms, CDFs, Q-Q plots.
 - **Consider outliers:** They can reveal errors or special cases; don't ignore without understanding.
 - **Account for noise:** Always measure confidence (CIs, p-values).
 - **Look at raw examples:** Validate that abstractions align with reality.
 - **Slice data:** Break into subgroups (browser, device, locale); check for consistency and mix shifts (Simpson's paradox).
 - **Practical vs. statistical significance:** Ask if differences actually matter.
 - **Check over time:** Look for consistency and investigate anomalies.
 - **Acknowledge filtering:** Specify filters and count excluded data.
 - **Clarify ratios:** Be explicit about numerator/denominator definitions.
-

Process: How to Approach Analysis

- **Separate stages:**
 1. **Validation** – is the data correct/consistent?
 2. **Description** – objective patterns/trends.
 3. **Evaluation** – does it mean something useful?
 - **Confirm setup:** Understand experiment/data collection context.
 - **Check invariants:** Ensure unrelated metrics didn't change.
 - **Start with standard metrics:** Validate new metrics against trusted ones.

- **Measure multiple ways:** Cross-validate metrics with alternative sources.
 - **Check reproducibility:** Should hold across populations/time.
 - **Compare with past measurements:** Large deviations likely indicate errors.
 - **Test new metrics on old data:** Build confidence before applying to new features.
 - **Form hypotheses, test with evidence:** Look for confirming/invalidating signals.
 - **Iterate end-to-end quickly:** Don't perfect early steps in exploratory work.
 - **Beware feedback loops:** Metrics fed into systems can't evaluate those systems.
-

Mindset: How to Work and Communicate

- **Start with questions, not tools or data:** Avoid aimless analysis.
 - **Be both skeptic and champion:** Validate insights and try to disprove them.
 - **Correlation $\neq$ causation:** Always test causal claims.
 - **Share with peers first:** Get critical feedback before showing consumers.
 - **Expect ignorance & mistakes:** Admitting uncertainty builds credibility.
-

Closing Thoughts

- Good analysis requires unseen careful work (filters, consistency checks, validation).
- It often takes longer than outsiders expect.
- Analysts must also educate consumers on why rigor matters.
- Choose tools suited to the data, not just the ones you're comfortable with.

1.6 Deep Learning Tuning Play-book:

Guide for starting a new project:

1. Choose the Model Architecture

- **Reuse, don't build from scratch:** Start with a well-established, popular model architecture that is known to work on a problem similar to yours.
- **Find a codebase:** Look for a documented codebase for a similar problem and reproduce that model as your starting point.
- **Definitions:**
 - **Model Architecture:** The framework for converting inputs to predictions (e.g., a neural network with defined layers).
 - **Model:** An architecture + specific parameter values.
 - **Model Family:** A template for building an architecture from hyperparameters.
- **Goal:** Choose a set of models to explore by tuning the model's hyperparameters.

2. Choose the Optimizer

- **Start with the most popular optimizer** for your problem type (e.g., Adam, SGD with momentum).
- **Initial Simplicity:** At the start of a project, choose an optimizer with few tunable hyperparameters to simplify the tuning process. Good choices are:
 - SGD with fixed momentum.
 - Adam with fixed epsilon, beta1, and beta2.
- **Later Tuning:** In later project stages, you can switch to a more general optimizer and tune its hyperparameters.
- **Beware:** Optimizers with more hyperparameters (like Adam) require more tuning effort.

3. Choose the Batch Size

- **Primary Goal: Minimize training time**, not to directly tune validation performance.
- **Key Rule of Thumb:** Use the **largest batch size your hardware can support** without causing a throughput bottleneck (where increasing the batch size no longer increases processing speed).

- **Why a large batch size?** It decreases the number of training steps needed to achieve a goal, which speeds up the overall training process (assuming you retune other hyperparameters).
- **Important Caveats:**
 - **Retuning Required:** Changing the batch size requires you to **retune most other hyperparameters** (especially learning rate and regularization).
 - **Resource Consideration:** Weigh the upfront cost of enabling larger batches (e.g., multi-GPU code) against the benefit of faster training cycles.
 - **Avoid Gradient Accumulation:** It simulates a larger batch size but does not provide the training throughput benefits of a true larger batch.

4. Choose the Initial Configuration

- **Guiding Principle:** Find a **simple, fast, low-resource** configuration that achieves **reasonable performance** (much better than random chance).
- **Simplicity:** Start without advanced features like custom regularizers.
- **Training Steps:** Choose a number of steps that balances performance (more steps are better) with tuning efficiency (fewer steps let you experiment faster). It's easier to start with a feasible budget and increase it later.

A Scientific approach to Improving Model Performance:

Hyperparameter Tuning Framework

1. Types of Hyperparameters

- **Scientific hyperparameters:** Central to the experimental question (e.g., optimizer choice, number of layers, whether to use dropout).
- **Nuisance hyperparameters:** Secondary, tuned to fairly compare scientific hyperparameters (e.g., learning rate, momentum).
- **Fixed hyperparameters:** Chosen once and kept constant (e.g., batch size, evaluation metric).
- **Conditional hyperparameters:** Exist only for certain scientific hyperparameter values (e.g., Adam $\rightarrow$ {learning_rate, beta1, beta2}, Nesterov $\rightarrow$ {learning_rate, momentum}).

Key insight: **Conditional hyperparameters with the same name (e.g., learning_rate) are not equivalent across algorithms.**

2. Designing Studies

- A **study** = a set of trials (hyperparameter configurations).
- To design a study, specify:
 - Which hyperparameters vary.
 - The search space (ranges/choices).
 - The number of trials.
 - The search algorithm (manual, quasi-random, Bayesian, evolutionary, etc.).

Study Goals

1. Compare different scientific hyperparameters.
2. Optimize over nuisance hyperparameters to make comparisons fair.

Approaches

- **Simple case:** Separate study per scientific hyperparameter.
- **Complex case:** Include scientific + nuisance hyperparameters in one search space (requires careful handling of conditional hyperparameters).

Preferred search strategies:

- **Exploration phase:** Quasi-random search (uniform coverage, robust with conditional hyperparams).
- **Exploitation phase:** Bayesian optimization (efficient refinement near best observed regions).

3. Balancing Resources

Tuning must balance:

1. **Breadth** → comparing enough scientific hyperparameters.
2. **Coverage** → wide ranges for nuisance hyperparameters.
3. **Density** → enough samples per search space region.

Constraints: limited compute, risk of unfair comparisons if one dimension is under-sampled.

4. Analyzing Experimental Results

Checklist after running studies:

- Is the **search space wide enough**? (Best points near boundary → expand range.)
- Were **enough trials sampled**? (Sparse → rerun with more trials.)
- What fraction of trials are **infeasible**? (If many → reparameterize or shrink space.)
- Any **optimization issues**? (divergence, instability).
- What do **training curves** show?
 - Overfitting? (add/tune regularization).
 - Variance across steps? (increase batch size, LR decay, Polyak averaging).
 - Still improving at end? (increase steps).
 - Saturated early? (reduce steps).

Visualization tools:

- **Hyperparameter axis plots** → performance vs. one hyperparameter, detect bad boundaries.
- **Isolation plots** → compare scientific hyperparameters after “optimizing away” nuisance ones.
- **Loss curves** → detect overfitting, instability, bugs.

Automate these plots → ensures consistent analysis.

5. Decision Making

- Recognize **variance sources**:
 - Trial variance (different seeds).
 - Study variance (different random search samples).
 - Data variance (different splits).
- Before adopting changes:
 - Run best trial multiple times to estimate variance.

- Only adopt changes that provide **consistent, meaningful improvements**.
 - Avoid adopting complexity unless improvement outweighs it.
-

6. Final Optimization Phase

- Once exploration identifies good ranges and essential hyperparameters:
 - Define a refined search space.
 - Use **Bayesian optimization** (or similar) for final tuning.
 - Handle divergent points properly (mark infeasible).

Determine the number of steps for each training run:

Training Workloads: Compute-Bound vs Not Compute-Bound

1. Compute-Bound Training

- **Definition:** Limited by compute budget (time/resources), not by data or convergence.
- **Key property:** Training/validation loss keeps improving if you train longer.
- **Implication:** Speeding up training = directly improving results.

Strategy

- Don't always train as long as possible—balance between:
 - Running more short experiments (faster feedback, better tuning).
 - Running fewer long experiments (closer to final results).
- **Round-based tuning recommended:**
 - **Round 1:** Many short runs → find promising hyperparameters.
 - **Round 2:** Few long runs → refine and finalize.
- **Pitfall:** Carrying over a short-run learning rate schedule → too small LR for most of a long run.
- **Adjustment for Round 2:** Extend time at high LR before decay (linear: lengthen plateau; cosine: keep base LR, extend total steps).

- **Hyperparameter transferability** (from short → long runs):
 - Likely: warmup length, initialization.
 - Maybe: architecture, optimizer, augmentation, regularization.
 - Unlikely: learning rate decay schedule.
-

2. Not Compute-Bound Training

- **Definition:** Compute is not the bottleneck—you can train “as long as you want.”
- **Key property:** Longer training may not improve validation loss (risk of overfitting).

Strategy

- Train until validation performance plateaus (use retrospective checkpointing).
 - Evaluation metrics should never degrade if checkpoints are chosen correctly.
 - **Never tune max_train_steps in a study:**
 - Fix a value across trials.
 - Use checkpoint analysis to refine later.
 - **How to pick initial max_train_steps:**
 1. Find a configuration that “perfectly fits” the training set (low loss, ~zero training error).
 2. Call its step count **N**.
 3. Run a **learning rate sweep** (no regularization/augmentation) with max N steps.
 4. The fastest trial to reach “perfect fit” → initial guess for max_train_steps.
 - Adjust based on gradient noise (augmentation, dropout → increase steps).
 - If optimizer/schedule improves → may need fewer steps.
-

3. Gradient Variance and Training Length

Factors that increase gradient variance (thus requiring **more steps to reach same validation loss**):

- Small batch sizes.
- Data augmentation.

- Regularization (e.g., dropout).
-

4. Round-Based Tuning Framework

- **Round 1 (short runs):** Explore many configs; useful for instability, initialization, warmup length, architecture insights.
- **Round 2 (long runs):** Apply best configs; tune learning rate schedule carefully; finalize model.
- **Optional further rounds:** Only if production runs are very expensive and team has mature tuning pipelines.
- **Strategy:** Exponentially increase run length across rounds to progressively derisk choices.

Additional Guidance For the training pipeline:

Evaluating Model Performance

1. Evaluation Settings

- **Online evaluation:** Measure metrics in production (most realistic).
- **Offline evaluation:** Use held-out train/val/test sets representative of production (may be expensive).
- **Periodic evaluations:** Run during training on a sampled subset; cheaper, faster, but only a proxy.

Best practice: use **periodic evaluations at fixed step intervals** (not time intervals).

2. Periodic Evaluations

Why:

- Monitor training progress.
- Enable retrospective checkpoint selection.
- Produce interpretable training curves.

Implementation details:

- Run training & eval on same instance (alternate between them).

- Use **larger batch size for eval** (no need to store activations).
 - Evaluate **every N steps**, not every T seconds (avoids misinterpretation if jobs are preempted).
 - **Check for periodicity in metrics** (could indicate shuffling bugs or data leakage).
 - Handle **partial batches** correctly → weight padded examples as zero.
 - **Save predictions** (not just metrics) → invaluable for debugging.
 - Save enough artifacts (e.g., SavedModels) to support offline analysis.
-

3. Sampling for Periodic Evaluation

- **Sample size trade-off:**
 - Small enough to run quickly.
 - Large enough for stable metrics & to avoid overfitting to validation over repeated runs.
 - **Check for skew:** Sampled-set performance should match full evaluation set.
 - **Imbalanced datasets:**
 - Track per-class metrics.
 - For rare classes, log counts of correctly classified examples to avoid overinterpreting tiny changes.
-

4. Checkpointing & Retrospective Selection

- Save checkpoints regularly (resilient to preemptions).
 - Best checkpoint is rarely the last one.
 - **Keep top N checkpoints** during training.
 - After training, select the best checkpoint retrospectively.
 - **Prospective early stopping usually unnecessary** → just run full trial budget + checkpoint selection.
-

5. Experiment Tracking

- Track all runs — “untracked experiments might as well not exist.”
 - Minimum columns in tracking sheet/system:
 - Study name.
 - Config location (link).
 - Notes/description.
 - Number of trials.
 - Best validation checkpoint performance.
 - Reproduction notes/commands.
-

6. Batch Normalization Gotchas

- **LayerNorm** often preferable → avoids many issues.
 - If using BatchNorm:
 - Multi-device = stats differ per device unless synchronized.
 - Ghost BatchNorm → use ~64 examples for stats, can work better.
 - Must subsample if per-device batch size > virtual batch size.
 - **EMAs for BN stats**: synchronize across devices before checkpointing. Many libraries only save EMAs from device 0 → bug risk.
-

7. Multi-Host Training Considerations

Bugs are easy to introduce—pay attention to:

- **Logging & checkpointing**: only one host should do it.
- **BN statistics**: synchronize across hosts before eval/checkpoint.
- **Data**: shard across hosts for efficiency.
- **RNG seeds**:
 - Same across hosts for model init.
 - Different across hosts for data shuffling/preprocessing.

Quasi-random Search:

Quasi-random search is a highly effective method for the initial **exploration phase** of hyperparameter tuning. It works by uniformly and non-adaptively sampling a search space using a mathematically defined sequence (like Halton) to spread out points more efficiently than random search.

Key Advantages Over Sophisticated Methods (e.g., Bayesian Optimization):

- **Non-adaptive & Flexible Analysis:** Because the points are chosen in advance, you can change your evaluation metric (e.g., from best validation error to final training error) *after* the experiments are run without rerunning any trials.
- **Statistically Reproducible:** The results are consistent and do not depend on the version of the algorithm, making studies easy to reproduce later.
- **Easier to Interpret:** The uniform exploration makes it simple to reason about the results. For example, if the best parameters are at the edge of the search space, it's a clear signal to adjust the bounds.
- **Robust to Parallelism:** Running trials in parallel or sequentially produces the same statistical results, which is not true for adaptive methods.
- **Simple and Effective:** It is simple to implement and works exceptionally well when many trials are run in parallel. It is often very difficult for adaptive algorithms to beat a quasi-random search with twice the budget under high parallelism.

When to use something else: Bayesian optimization becomes more attractive only when you can run very few trials in parallel but can afford to run many in sequence, despite making results harder to interpret.

Implementation: The text mentions implementations are available in **Open-Source Vizier** (using `algorithm="QUASI_RANDOM_SEARCH"`) and other linked examples. If unavailable, pseudo-random search is an acceptable, though slightly less efficient, substitute.

1.7 Introduction to Responsible AI:

Develop and scale AI responsibly

Artificial intelligence (AI) powers many apps and services that people use in daily life. With billions of AI users across fields from business to healthcare to education, it is critical that companies work to ensure the benefits of these technologies far outweigh the harms.

Responsible AI considers the societal impact of the development and scale of these technologies, including potential harms and benefits. Google's AI Principles provide a framework to create the most helpful, safe, and trusted experiences for all. The principles include objectives for AI applications as well as applications we will not pursue in the development of AI systems.

Fairness:

Fairness addresses the possible disparate outcomes end users may experience related to sensitive characteristics such as race, income, sexual orientation, or gender through algorithmic decision-making. For example, might a hiring algorithm have biases for or against applicants with names associated with a particular gender or ethnicity?

Accountability

Accountability means owning responsibility for the effects of an AI system. Accountability typically involves **transparency**, or sharing information about system behavior and organizational process, which may include documenting and sharing how models and datasets were created, trained, and evaluated. The following sites explain two valuable modes of accountability documentation:

- Model Cards
- Data Cards

Another dimension of accountability is **interpretability**, which involves the understanding of ML model decisions, where humans are able to identify features that lead to a prediction. Moreover, **explainability** is the ability for a model's automated decisions to be explained in a way for humans to understand.

Safety

AI **safety** includes a set of design and operational techniques to follow to avoid and contain actions that can cause harm, intentionally or unintentionally. For example, do AI systems behave as intended, even in the face of a security breach or targeted attack? Is the AI system robust enough to operate safely even when perturbed? How do you plan ahead to prevent or avoid risks? Is the AI system reliable and stable under pressure?

Privacy

Privacy practices in Responsible AI involve the consideration of potential implications in using sensitive data. This includes not only respecting legal and regulatory requirements, but also considering social norms and typical individual expectations. For example, what safeguards need to be put in place to ensure the privacy of individuals, considering that ML models may remember or reveal aspects of the data that they have been exposed to? What steps are needed to ensure users have adequate transparency and control of their data?

1.8 Adversarial Testing for Generative AI:

What is Adversarial Testing?

Adversarial testing is the practice of deliberately trying to “break” an AI system by giving it inputs designed to provoke unsafe, harmful, or problematic outputs. Unlike standard evaluation, it focuses on failure modes and edge cases, not average-case performance.

- **Explicit adversarial queries:** Contain obvious policy-violating content (e.g., harmful, offensive, trick attempts).
 - **Implicit adversarial queries:** Seem harmless but touch on sensitive topics (e.g., health, religion, demographics).
-

Why it's important

- Identifies weaknesses, unsafe outputs, and policy violations.
 - Guides improvements through fine-tuning, safeguards, or filters.
 - Helps assess readiness for product launch by quantifying risks.
-

Example Workflow

1. Identify Inputs

- Base test scope on product policies, known failure modes, use cases, and edge cases.
- Ensure diversity in query style (length, vocabulary, phrasing) and topics (sensitive, global contexts).

2. Find or Create Test Datasets

- Use existing datasets (benchmarks, policy violation corpora) for baseline coverage.
- Create seed datasets manually and expand using data synthesis tools.
- Ensure queries resemble real-world use but are adversarially designed.

3. Analyze Datasets

- Check diversity (lexical + semantic), policy coverage, uniqueness, and adversarial quality.

4. Generate & Annotate Outputs

- Run model on test datasets.
- Use automated safety classifiers where possible, but validate with human raters.
- Categorize failures by type and harm level.

5. Report & Mitigate

- Summarize results with metrics, visualizations, and failure examples.
- Use findings to improve the model (filters, blocklists, fine-tuning).
- Share insights with stakeholders for informed decisions.

In short: Adversarial testing is a structured process to probe AI systems with high-risk queries, analyze failures, and implement safeguards to make models safer and more reliable

2. Self-Directed Exploration:

2.1 Unsloth for Finetuning Large Language Models:

What Is Unsloth?

Unsloth is an open-source framework for efficient fine-tuning and reinforcement learning of large language models (LLMs). It's built atop Hugging Face's ecosystem but implements custom, highly optimized GPU kernels (written in Triton) to make training significantly faster and more memory-efficient.

Unsloth allows u to fine tune LLMs effectively on a very small dataset and ensure very fast training.

What Is It Used For?

- **Fine-Tuning LLMs:** Tailor general pretrained models (e.g., Llama, Mistral, Gemma, gpt-oss) to specific tasks, domains, or behaviors.
- **Resource-Constrained Training:** Run fine-tuning even on modest GPUs—3 GB VRAM in Colab/Kaggle setups.
- **Broader ML Support:** Beyond LLMs, Unsloth supports TTS (text-to-speech), STT, vision, diffusion models, BERT-style tasks, reinforcement learning (e.g., DPO, ORPO), and continued pretraining.

How Effective Is It?

- **Speed:** Up to **2× faster** than standard Transformers libraries—and up to **30× faster** compared to FlashAttention 2 (FA2) when using multiple GPUs. Even on a single GPU, Unsloth is significantly quicker.
- **Memory Efficiency:** Uses **70–90% less VRAM** than conventional methods.
- **Longer Context Windows:** Supports context lengths far exceeding standard libraries.

- Example with Llama 3.1 (8B) on 16 GB GPU: Unsloth handles ~40 k tokens vs ~2.5 k tokens by Hugging Face + FA2.
 - With 80 GB VRAM: ~342 k tokens vs ~28.5 k.
 - **Accuracy:** Maintains full numerical exactness—no approximations or accuracy loss despite quantization.
-

What Unique Advantages Does It Offer for Fine-Tuning LLMs?

1. Optimized GPU Kernels

Unsloth hand-writes compute-heavy operations in Triton rather than relying on general libraries—this underpins its major performance and memory advantages.

2. Dynamic 4-bit Quantization (“Unsloth Dynamic Quants”)

These smart quantization methods selectively avoid quantizing certain parameters, achieving near-floating-point accuracy with only a small VRAM penalty.

3. Flexible Fine-Tuning Modes

Supports standard LoRA, QLoRA, full fine-tuning, and reinforcement learning methods like DPO or ORPO—all within one unified interface.

4. Extremely Long Context Support

Greatly extended context windows (multiple hundreds of thousands of tokens) enable tasks like long-document comprehension or reasoning that are unattainable with standard tools.

5. Ease of Use

- Beginner-friendly notebooks make getting started easy—even on Colab or Kaggle with limited resources.
- Simple API and integration with Hugging Face make workflows smoother and more accessible.

Instruct or Base Model?

When preparing for fine-tuning, one of the first decisions you'll face is whether to use an instruct model or a base model.

Instruct Models

Instruct models are pre-trained with built-in instructions, making them ready to use without any fine-tuning. These models, including GGUFs and others commonly available, are optimized for direct usage and respond effectively to prompts right out of the box. Instruct models work with conversational chat templates like ChatML or ShareGPT.

Base Models

Base models, on the other hand, are the original pre-trained versions without instruction fine-tuning. These are specifically designed for customization through fine-tuning, allowing you to adapt them to your unique needs. Base models are compatible with instruction-style templates like [Alpaca or Vicuna](#), but they generally do not support conversational chat templates out of the box.

Should I Choose Instruct or Base?

The decision often depends on the quantity, quality, and type of your data:

- **1,000+ Rows of Data:** If you have a large dataset with over 1,000 rows, it's generally best to fine-tune the base model.
- **300–1,000 Rows of High-Quality Data:** With a medium-sized, high-quality dataset, fine-tuning the base or instruct model are both viable options.
- **Less than 300 Rows:** For smaller datasets, the instruct model is typically the better choice. Fine-tuning the instruct model enables it to align with specific needs while preserving its built-in instructional capabilities. This ensures it can follow general instructions without additional input unless you intend to significantly alter its functionality.

Unsloth recommend starting with **Instruct models**, as they allow direct fine-tuning using conversational chat templates (ChatML, ShareGPT etc.) and require less data compared to **Base models** (which uses Alpaca, Vicuna etc). Learn more about the differences between instruct and base models here.

- Model names ending in **unsloth-bnb-4bit** indicate they are **Unsloth dynamic 4-bit quants**. These models consume slightly more VRAM than standard BitsAndBytes 4-bit models but offer significantly higher accuracy.

- If a model name ends with just **bnb-4bit**, without "unsloth", it refers to a standard BitsAndBytes 4-bit quantization.
- Models with **no suffix** are in their original **16-bit or 8-bit formats**. While they are the original models from the official model creators, we sometimes include important fixes - such as chat template or tokenizer fixes. So it's recommended to use our versions when available.

3. Challenges and Approaches:

1. The theoretical work for this week was a lot and just studying theoretically about something isn't enough and I always find theory a bit boring.
2. That's why I just got a basic understanding of the theoretical topics.

As there was very little information about Unsloth (Self-Directed Task) on youtube and other platform so to tackle this is studied about Unsloth from their official documentation and all confusions were cleared.

4. Result and Analysis:

Findings About Unsloth:-

- **Purpose:** Makes LLM fine-tuning/training faster, cheaper, and more accessible.
- **Effectiveness:** Up to **30× faster**, **70–90% less VRAM**, supports **very long context windows**, with **no accuracy loss**.
- **Unique Advantages:**
 - Dynamic 4-bit quantization (efficient + accurate)
 - Flexible fine-tuning (full, LoRA, QLoRA, partial, RLHF/DPO, continued pretraining)
 - Runs even on low-end GPUs (3–12 GB VRAM)
 - Supports LLMs + other models (BERT, vision, speech, diffusion)
- **Biggest Value:** Lowers hardware barriers while keeping performance high, making LLM fine-tuning practical for both researchers and practitioners.

Ways to Fine Tune a Model using Unsloth:

1. Full Fine-Tuning

- **What it is:** Update **all model parameters** (millions/billions of weights).
- **Pros:** Maximum control and adaptation, especially for highly domain-specific data.
- **Cons:** Expensive in VRAM/compute (e.g., 7B model may need > 48GB VRAM without optimization).
- **With Unsloth:** Thanks to custom Triton kernels + memory optimizations, this becomes feasible on smaller GPUs (24–48GB), which would otherwise be impossible with Hugging Face alone.

2. LoRA (Low-Rank Adaptation)

- **What it is:** Injects small trainable “adapter” layers while freezing most model weights.
- **Pros:** Very lightweight, efficient, and effective for many tasks.
- **Cons:** Limited capacity for deep domain shifts.

- **With Unsloth:** Runs even on consumer GPUs (12–16 GB VRAM). Often the go-to choice for instruction-tuning.
-

3. QLoRA (Quantized LoRA)

- **What it is:** Quantizes the base model (usually 4-bit) while training LoRA adapters.
 - **Pros:** Extreme VRAM savings — can fine-tune large models (7B–70B) on single GPUs with as little as 8–24 GB VRAM.
 - **Cons:** Quantization can introduce small overheads; still adapter-only.
 - **With Unsloth:** Implements **dynamic quantization** (Unsloth Dynamic Quants), which reduces accuracy loss compared to vanilla QLoRA.
-

4. Partial Fine-Tuning (Selective Layer Updates)

- **What it is:** Fine-tune only certain layers (e.g., embeddings, output layer, or last N transformer blocks).
 - **Pros:** Balances performance and efficiency.
 - **Cons:** Requires careful choice of which layers to unfreeze.
 - **With Unsloth:** Easy to configure which parameters are trainable.
-

5. Reinforcement Learning Fine-Tuning

- **Methods supported:**
 - **DPO (Direct Preference Optimization)**
 - **ORPO (Odds Ratio Preference Optimization)**
 - Others in experimental stages.
 - **Pros:** Aligns models with human preferences/feedback without costly RLHF pipelines.
 - **With Unsloth:** Optimized implementations that run on lower VRAM compared to traditional RLHF setups.
-

6. Continued Pretraining (Domain Adaptation)

- **What it is:** Keep training the base model on domain-specific unlabeled corpora before task fine-tuning.
 - **Pros:** Adapts language style and knowledge (e.g., medical, legal, or financial domains).
 - **With Unsloth:** Faster, longer-context training possible (hundreds of thousands of tokens).
-

7. Instruction / Supervised Fine-Tuning (SFT)

- **What it is:** Train on (prompt → response) pairs.
 - **Pros:** Core step in aligning base LLMs into chatbots, assistants, and domain-experts.
 - **With Unsloth:** Runs easily with LoRA/QLoRA or full fine-tuning depending on resources.
-

8. Multi-Modal / Other Model Types

Although mainly for LLMs, Unsloth also supports:

- BERT-style encoders (for classification, retrieval).
 - Vision + diffusion models.
 - TTS (text-to-speech) and STT (speech-to-text).
- This makes it a broader optimization toolkit, not just for text models.

5. Conclusion:

For this week majority of tasks were theoretical and I understand about adopting best practices while solving a particular problem such as how to select different evaluation metrics for different tasks.

As for Unsloth I covered the theoretical aspect and I am strongly motivated towards fine tuning LLMs on different datasets.

Unsloth provides a ready to go Colab notebooks and we just have to load our dataset and do some changes in the prompt and start the training and through-out training and after training is complete we have to see the logs and evaluation metrics and adjust hyperparameters accordingly.

I explored some datasets from hugging face that can be used for fine tuning. One example is the link below.

<https://huggingface.co/datasets/lextale/FirstAidInstructionsDataset>

6. References:

- <https://www.kaggle.com/datasets>
- <https://huggingface.co/datasets>
- <https://huggingface.co/datasets/lextale/FirstAidInstructionsDataset>
- <https://unsloth.ai/>
- <https://docs.unsloth.ai/>